

1. TLB / Page Table / Cache Possibility Table (10 pts)

The lookup order is TLB → Page Table → Cache.

TLB	Page Table	Cache	Possible?	Under what circumstances?
Hit	Hit	Hit	Yes	Normal case – TLB has the translation, page is in memory, and the data is in cache.
Hit	Hit	Miss	Yes	TLB has the translation and the page is valid, but the data isn't in the cache (compulsory/capacity/conflict miss).
Miss	Hit	Hit	Yes	TLB doesn't have the entry (maybe evicted earlier), but the page table has a valid mapping. After getting the physical address, the data is found in cache.
Miss	Hit	Miss	Yes	TLB misses, page table has the mapping, but the data block isn't in cache. Need to go to main memory.
Miss	Miss	Miss	Yes	TLB misses, page table also misses (page fault – page isn't in physical memory). After handling the page fault and bringing the page from disk, the data won't be in cache either.
Hit	Miss	Hit/Miss	No	A TLB hit means we have a valid translation, which means the page <i>must</i> be in physical memory. So the page table can't miss – TLB hit implies page table hit.
Miss	Miss	Hit	No	If the page table misses, that's a page fault – the page isn't in physical memory. So there's no way the data is in the cache since cache holds physical memory data.

2. Data Cache Simulation (50 pts)

Cache starts with word addresses 0–15. Reference string:

1, 20, 2, 3, 4, 18, 5, 19, 33, 34, 1, 4

(a) Direct-Mapped, 1-Word Blocks, 16 Blocks

Index = Word Addr mod 16. Initially block i holds word address i .

#	Addr	Index	H/M
1	1	1	Hit
2	20	4	Miss (had 4, replace w/ 20)
3	2	2	Hit
4	3	3	Hit
5	4	4	Miss (had 20, replace w/ 4)
6	18	2	Miss (had 2, replace w/ 18)
7	5	5	Hit
8	19	3	Miss (had 3, replace w/ 19)
9	33	1	Miss (had 1, replace w/ 33)
10	34	2	Miss (had 18, replace w/ 34)
11	1	1	Miss (had 33, replace w/ 1)
12	4	4	Hit

Hits = 5, Misses = 7

Hit rate = $5/12 \approx 41.67\%$

Final cache state:

Idx	Addr	Idx	Addr
0	0	8	8
1	1	9	9
2	34	10	10
3	19	11	11
4	4	12	12
5	5	13	13
6	6	14	14
7	7	15	15

(b) Direct-Mapped, 2-Word Blocks, 8 Blocks

Block # = $\lfloor \text{Addr}/2 \rfloor$, Index = Block # mod 8. Each block holds words $\{2k, 2k+1\}$.

Initially loaded with addrs 0–15, so indices 0–7 hold blocks 0–7.

#	Addr	Blk	Idx	H/M
1	1	0	0	Hit (blk 0: {0,1})
2	20	10	2	Miss (had blk 2, replace w/ {20,21})
3	2	1	1	Hit (blk 1: {2,3})
4	3	1	1	Hit (still blk 1)
5	4	2	2	Miss (had blk 10, replace w/ {4,5})
6	18	9	1	Miss (had blk 1, replace w/ {18,19})
7	5	2	2	Hit (blk 2: {4,5})
8	19	9	1	Hit (blk 9: {18,19})
9	33	16	0	Miss (had blk 0, replace w/ {32,33})
10	34	17	1	Miss (had blk 9, replace w/ {34,35})
11	1	0	0	Miss (had blk 16, replace w/ {0,1})
12	4	2	2	Hit (blk 2: {4,5})

Hits = 6, Misses = 6

Hit rate = $6/12 = 50\%$

Final cache state:

Idx	Blk #	Words
0	0	{0, 1}
1	17	{34, 35}
2	2	{4, 5}
3	3	{6, 7}
4	4	{8, 9}
5	5	{10, 11}
6	6	{12, 13}
7	7	{14, 15}

(c) 2-Way Set-Associative, 2-Word Blocks, 8 Blocks Total, LRU

8 blocks / 2 ways = 4 sets. Block # = $\lfloor \text{Addr}/2 \rfloor$, Set = Block # mod 4.

Initial state (addrs 0–15 loaded in order, so blocks 0–3 are LRU, blocks 4–7 are MRU):

Set	Way 0 (LRU)	Way 1 (MRU)
0	Blk 0: {0,1}	Blk 4: {8,9}
1	Blk 1: {2,3}	Blk 5: {10,11}
2	Blk 2: {4,5}	Blk 6: {12,13}
3	Blk 3: {6,7}	Blk 7: {14,15}

#	Addr	Blk	Set	H/M	Set After (LRU → MRU)
1	1	0	0	Hit	Blk4:{8,9}, Blk0:{0,1}
2	20	10	2	Miss	Evict Blk2 → Blk6:{12,13}, Blk10:{20,21}
3	2	1	1	Hit	Blk5:{10,11}, Blk1:{2,3}
4	3	1	1	Hit	Blk5:{10,11}, Blk1:{2,3}
5	4	2	2	Miss	Evict Blk6 → Blk10:{20,21}, Blk2:{4,5}
6	18	9	1	Miss	Evict Blk5 → Blk1:{2,3}, Blk9:{18,19}
7	5	2	2	Hit	Blk10:{20,21}, Blk2:{4,5}
8	19	9	1	Hit	Blk1:{2,3}, Blk9:{18,19}
9	33	16	0	Miss	Evict Blk4 → Blk0:{0,1}, Blk16:{32,33}
10	34	17	1	Miss	Evict Blk1 → Blk9:{18,19}, Blk17:{34,35}
11	1	0	0	Hit	Blk16:{32,33}, Blk0:{0,1}
12	4	2	2	Hit	Blk10:{20,21}, Blk2:{4,5}

Hits = 7, Misses = 5

Hit rate = $7/12 \approx 58.33\%$

Final cache state:

Set	Way 0 (LRU)	Way 1 (MRU)
0	Blk 16: {32,33}	Blk 0: {0,1}
1	Blk 9: {18,19}	Blk 17: {34,35}
2	Blk 10: {20,21}	Blk 2: {4,5}
3	Blk 3: {6,7}	Blk 7: {14,15}

3. AMAT and CPI (20 pts)

Given:

- \$I cache: 90% hit rate, 1 cycle hit time, 100 cycle miss penalty
- \$D cache: 85% hit rate, 2 cycle hit time, 120 cycle miss penalty
- Memory instructions (lw/sw) are 30% of the instruction mix
- Base CPI = 1.0 (perfect memory)

(a) AMAT

AMAT = Hit Time + Miss Rate $\times$ Miss Penalty

\$I cache:

$$\text{AMAT}_I = 1 + 0.10 \times 100 = 11 \text{ cycles}$$

\$D cache:

$$\text{AMAT}_D = 2 + 0.15 \times 120 = 20 \text{ cycles}$$

(b) CPI with Stalls

\$I cache stall (every instruction accesses \$I):

$$\text{Stall}_I = 0.10 \times 100 = 10 \text{ cycles/instruction}$$

\$D cache stall (only 30% of instructions access \$D):

$$\text{Stall}_D = 0.30 \times 0.15 \times 120 = 5.4 \text{ cycles/instruction}$$

Overall:

$$\text{CPI} = 1.0 + 10 + 5.4 = \mathbf{16.4}$$

4. Cache Optimization Strategies (20 pts)

(a) Graph Applications (irregular access patterns)

Graph workloads like traversing a Twitter follower graph have random, pointer-chasing access patterns with poor spatial and temporal locality.

- **Higher associativity** – Random accesses cause lots of conflict misses. Using higher associativity (like 8-way or fully associative) helps reduce these since different addresses won't keep evicting each other from the same set.
- **Smaller block size** – Spatial locality is bad here, so large cache lines just waste bandwidth fetching neighboring data that won't be used. Smaller blocks are more efficient.
- **Non-blocking caches** – Graph traversals can have many independent memory accesses happening. A non-blocking cache lets multiple misses overlap so the processor isn't just sitting idle waiting for one miss at a time.
- **Disable stride prefetching** – Standard hardware prefetchers assume sequential/stride patterns, which doesn't apply here. They'd just pollute the cache with wrong data, so it's better to turn them off for this kind of workload.
- **Victim cache** – A small fully-associative victim cache can catch blocks that just got evicted and might be needed again soon (like revisiting a graph node), which helps with conflict misses without much extra cost.

(b) Streaming Applications (regular access patterns)

Image/video processing scans through data sequentially with very predictable patterns – great spatial locality but low temporal locality since each piece of data is usually only used once.

- **Larger block size** – Sequential access means neighboring data *will* be used, so bigger cache lines take advantage of the spatial locality and reduce the miss rate.
- **Hardware prefetching** – The regular stride pattern is exactly what hardware prefetchers are designed for. They can detect the pattern and bring data into the cache ahead of time, hiding memory latency.
- **Lower associativity** – Since the access pattern is predictable and doesn't cause many conflict misses, a direct-mapped or 2-way cache is fine. This gives faster hit times and uses less power.
- **Write buffer** – Streaming apps that produce output (like processed video frames) generate lots of writes. A write buffer absorbs these so the processor doesn't stall waiting for each write to finish.
- **Cache bypassing / stream buffer** – If the data is truly only accessed once, it can bypass the cache entirely (or go through a stream buffer) so it doesn't pollute the cache and evict other useful data.